

Vector Data, Distance, and IVs

ECON 895 Spatial Techniques in Empirical Economics

Noel D. Johnson
George Mason University

Week 2 August 31, 2026

njohnsoL@gmu.edu

Plan for tonight

Why R

The tidyverse detour

Simple features and vector data

Projections, distance, and area

Distance as an instrument

We break around 8:45.

Why R

Introduction

There are many ways to handle spatial data, ArcGIS and QGIS and GRASS among them. In this course we do all of our spatial computation in R. Why is that a good idea?

- ArcGIS is proprietary, expensive, slow, and buggy. You have to learn both Arc and Python to use it intelligently.
- QGIS is excellent but is built around clicking, pointing, and mapping. Most of its spatial operations are powered by the same C++ libraries R calls.
- R is free, open source, and driven by a large community of developers. There are more than a hundred packages in r-spatial today.
- With R you create, modify, and analyze spatial data in one place. And you can run your regressions there too.

The reason that matters most

Reproducibility

A script is a complete, rerunnable record of every step from raw data to published table. A sequence of clicks is not.

- You will not remember in March what you clicked in October.
- A referee asking how a variable was built needs an answer better than a description of a menu.
- **And, new this year.** When a model writes the code, the script is also the artifact you audit. There is nothing to check in a click.

Almost every number you are about to see on these slides was produced by running the code on the slide at the moment the deck was built. That is not a flourish. It is the standard this course asks of you.

Spatial data types

There are “only” two types of spatial data.

Vector data (tonight and next week)

- Points
- Lines
- Polygons

Raster data (Week 4)

- Continuous
- Categorical or binary

Before we can work with vector data, we have to take a detour through the tidyverse.

The tidyverse detour

Why not base R?

- Many base R functions were written twenty or thirty years ago. Changing them breaks existing code, so most innovation happens in new packages.
- We work with the **tidyverse**, a collection of packages following one consistent paradigm. Its advantage here is that it makes **simple features**, the modern representation of vector data in `sf`, considerably easier to handle.

These are the versions this deck was built against.

```
packageVersion("dplyr"); packageVersion("sf")
```

```
## [1] '1.1.4'
```

```
## [1] '1.0.24'
```

Use `library()` in scripts, not `require()`. `library()` fails loudly when the package is missing, which is what you want.

A tibble is a data frame with better manners

Every tibble is a data frame. Not every data frame is a tibble. The difference that matters tonight is that a tibble prints the first ten rows and only the columns that fit, so an accidental `print()` of a large object no longer floods the console.

```
cps08 <- read_csv("../data/cps08.csv", show_col_types = FALSE)
class(cps08)
```

```
## [1] "spec_tbl_df" "tbl_df"      "tbl"         "data.frame"
```

```
print(cps08, n = 3)
```

```
## # A tibble: 7,711 x 5
##   ahe   year bachelor female   age
##   <dbl> <dbl> <chr>    <chr> <dbl>
## 1 38.5   2008 yes      no     33
## 2 12.5   2008 yes      no     31
## 3  9.86  2008 no       no     30
## # i 7,708 more rows
```

Subsetting with `[ ]` returns another tibble, even for a single column. Use `[[ ]]` or `$` when you want a vector. Write with `write_csv()`.

What is tidy data?

Tidy data imposes structure, the way Stata or a well kept spreadsheet does.

The figure consists of three panels illustrating different ways to view a dataset. Each panel shows a table with four columns: 'country', 'year', 'cases', and 'population'. The data rows are: Afghanistan (2000, 15, 121, 7071), Afghanistan (2000, 666, 20, 360), Brazil (1999, 3737, 172, 362), Brazil (2000, 8488, 174, 362), China (1999, 21258, 1272, 15272), and China (2000, 21266, 1280, 15272).

variables: This panel shows vertical double-headed arrows between columns, indicating that each variable (country, year, cases, population) has its own column.

observations: This panel shows horizontal double-headed arrows between rows, indicating that each observation (country-year combination) has its own row.

values: This panel shows circles around individual data points in each cell, indicating that each value must have its own cell.

country	year	cases	population
Afghanistan	2000	15	1217071
Afghanistan	2000	666	20360
Brazil	1999	3737	172362
Brazil	2000	8488	174362
China	1999	21258	127215272
China	2000	21266	128015272

1. Each variable must have its own column.
2. Each observation must have its own row.
3. Each value must have its own cell.

Figure from Wickham and Grolemund, *R for Data Science*.

Subsetting dplyr-style

`select()` takes columns. `filter()` takes rows.

```
select(cps08, age, ahe)
```

```
## # A tibble: 7,711 x 2
##   age   ahe
##   <dbl> <dbl>
## 1    33 38.5
## 2    31 12.5
## 3    30  9.86
## 4    30  8.24
## # i 7,707 more rows
```

```
filter(cps08, age == 33)
```

```
## # A tibble: 720 x 5
##   ahe   year bachelor female   age
##   <dbl> <dbl> <chr>   <chr> <dbl>
## 1  38.5  2008 yes      no     33
## 2  29.8  2008 yes      no     33
## 3  19.2  2008 yes      no     33
## 4  21.6  2008 yes      no     33
## # i 716 more rows
```

`dplyr` quotes the inputs for you, so the column names need no quotation marks.

`select()` has helpers such as `starts_with()` and `contains()`. Inside `filter()` the standard comparison operators work, and so do `is.na()` and `between()`. `slice(2:6)` takes rows by position rather than by condition.

run it

Sorting, renaming, and changing variables

Three more verbs. Notice what it costs to use them together.

```
tmp <- rename(cps08, wage = ahe)
tmp <- mutate(tmp, lwage = log(wage))
head(arrange(tmp, desc(lwage)), 3)
```

```
## # A tibble: 3 x 6
##   wage  year bachelor female   age lwage
##   <dbl> <dbl> <chr>    <chr> <dbl> <dbl>
## 1  82.4  2008  yes      no     30   4.41
## 2  76.9  2008  yes      no     34   4.34
## 3  75.9  2008  yes      no     29   4.33
```

`desc()` sorts descending. `transmute()` keeps only what it creates. Useful companions are `log()`, `lead()`, and `lag()`.

The pipe operator

That last slide needed a throwaway object called `tmp` and needed it twice. The pipe removes it.

```
cps08 %>%  
  rename(wage = ahe) %>%  
  mutate(lwage = log(wage)) %>%  
  arrange(desc(lwage)) %>%  
  select(age, wage, lwage) %>%  
  print(n = 3)
```

```
## # A tibble: 7,711 x 3  
##   age  wage lwage  
##   <dbl> <dbl> <dbl>  
## 1    30  82.4  4.41  
## 2    34  76.9  4.34  
## 3    29  75.9  4.33  
## # i 7,708 more rows
```

Read `%>%` as “and then”. It hands the left side to the first argument on the right, and where you need it in another position `.` stands in for it. That is why `st_area(.)` and `st_distance(., mainz)` read the way they do later tonight. Base R now has a native pipe, `|>`, which works the same way here. **Keep pipes short.** A ten step pipe is a ten step pipe you cannot debug, because there is no intermediate object left to look at, and that matters more when a model wrote the pipe rather than you.

Grouping and summarizing dplyr-style

Grouped statistics are cumbersome in base R. dplyr gives us verbs.

```
cps08 %>%  
  group_by(female, bachelor) %>%  
  summarize(mean_ahe = mean(ahe), .groups = "drop")
```

```
## # A tibble: 4 x 3  
##   female bachelor mean_ahe  
##   <chr>   <chr>      <dbl>  
## 1 no     no          16.6  
## 2 no     yes          25.0  
## 3 yes    no          13.2  
## 4 yes    yes          20.9
```

count(x, var) is this same pair of verbs in one call, for when all you want is how many.

Grouping is sticky

A group that survives a summarize() or a mutate() silently changes every calculation after it. Set .groups when you summarize, call ungroup() when you are done, and name your summary columns so you can see what you made.

The join that eats your data

A join looks harmless. Watch the row count.

```
lookup <- tibble(age = 25:30, cohort = "prime")  
  
nrow(cps08)                                # before  
joined <- inner_join(cps08, lookup, by = "age")  
nrow(joined)                                # after
```

- No error. No warning. **4,552** come back. Every row whose age falls outside 25 to 30 is gone. `left_join()`, `full_join()`, and `bind_rows()` can each change the row count too.
- **The habit.** Count rows on both sides of every join, and then look at what you kept. Last week's Tanzania join is why the second half of that sentence matters. The count alone looked exactly as plausible as the truth.

Getting data in and out

R has its own binary format, RDS. Store single objects rather than the whole workspace.

```
saveRDS(cps08, "./data/cps08.rds")    # saving  
cps08 <- readRDS("./data/cps08.rds")  # loading
```

haven reads Stata, SPSS, and SAS. readxl reads Excel.

```
library(haven); my_tbl <- read_dta("some_file.dta")  
library(readxl); my_tbl <- read_excel("some_file.xlsx", sheet = 1)
```

Use RDS when you will keep working in R and do not want to lose information about the object, such as grouping keys or, in an hour, a coordinate reference system. A CSV cannot carry a CRS.

Simple features and vector data

What are simple features?

A city is two numbers. A country border is a few thousand coordinates that have to stay in order, and the whole of it has to fit in one cell of one row, so that population and country name and everything else can sit beside it.

Simple features are the agreement about how to store that

One column holds an entire geometry per row. A spatial dataset is then a tibble with a geometry column, which is why `filter()` and `group_by()` go on working once it is loaded.

The agreement is written down because a border you save has to be readable by software you do not control and did not write. An industry consortium, the **OGC**, drafted it in the late 1990s, and the international standards body, **ISO**, published it as ISO 19125. That is the reason one shapefile opens in R, in QGIS, and in ArcGIS, and gives the same answer in each.

What is underneath sf

sf is a thin R wrapper around three C libraries.

```
sf_extSoftVersion()[c("GEOS", "GDAL", "PROJ")]
```

```
##      GEOS      GDAL      PROJ  
## "3.13.0"  "3.8.5"  "9.5.1"
```

- **GDAL**, the Geospatial Data Abstraction Library, reads and writes the file formats. 73 of them on this machine. It is what `st_read()` actually calls.
- **GEOS** does the geometry, so intersections and buffers and areas.
- **PROJ** does the projections. When a coordinate reference system goes wrong later tonight, PROJ is the thing complaining.

Worth knowing because the error messages come from these libraries and not from R, so they do not read like R errors, and because pinning versions means pinning theirs too.

Basic simple features

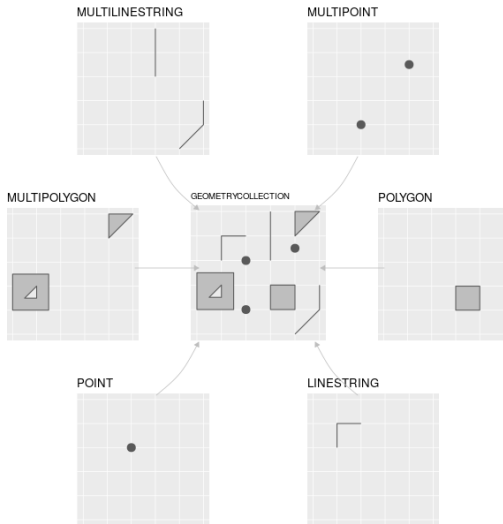


Figure from the `sf` package vignette.

Anatomy of an sf object

Geometries are native R objects but print as well known text, or WKT.

```
## Simple feature collection with 100 features and 6 fields
## geometry type:  MULTIPOLYGON
## dimension:      XY
## bbox:           xmin: -84.32385 ymin: 33.88199 xmax: -75.45698 ymax: 36.58965
## epsg (SRID):    4267
## proj4string:     +proj=longlat +datum=NAD27 +no_defs
## precision:       double (default; no precision model)
## First 3 features:
##   BIR74 SID74 NWBIR74 BIR79 SID79 NWBIR79 geom
## 1  1091     1      10  1364     0      19 MULTIPOLYGON((( -81.47275543...
## 2   487     0      10   542     3      12 MULTIPOLYGON((( -81.23989105...
## 3  3188     5     208  3616     6     260 MULTIPOLYGON((( -80.45634460...
```

Simple feature

Simple feature geometry list-column (sfc)

Simple feature geometry (sfg)

Green is a simple feature, one row of a tibble. **Red** is a simple feature list column, class `sfc`. **Blue** is a single geometry, class `sfg`.

Installing and loading sf

```
install.packages("sf")  
library(sf)
```

- One line, and it pulls in GEOS, GDAL and PROJ behind it. Those three are why installation sometimes fails on macOS and Linux rather than on Windows, and why the error text is rarely about R. Your setup script checks all three.

Loading vector data

`st_read()` and `st_write()` import and export most common vector formats. See `st_drivers()`.

```
library(tidyverse)
library(sf)

europe_raw <- st_read("./data/europe_modern.shp")  # read what it prints
```

The object you read is not yet the object you analyze. This one still needs `st_transform()`, which is why it is not called `europe_sf`.

Do not open a script with `rm(list = ls())` and a `setwd()`

The 2022 version of this slide did both. Use an RStudio Project so the working directory is the project root, and keep every path relative to it. Absolute paths are why other people cannot run your code.

Learning about this data I

```
class(cities_sf)           # what is it?
```

```
## [1] "sf"           "spec_tbl_df" "tbl_df"      "tbl"
## [5] "data.frame"
```

```
st_crs(cities_sf)$epsg     # what is the CRS?
```

```
## [1] 4326
```

```
st_bbox(cities_sf)         # and the extent?
```

```
##      xmin      ymin      xmax      ymax
## -9.699932 36.014321 18.623796 63.430515
```

st_crs() is the second thing you run on any object you load. Not the tenth. The whole course turns on knowing the answer before you measure anything.

Learning about this data II

```
dim(cities_sf); names(cities_sf)  # note the geometry column
```

```
## [1] 1801    5
```

```
## [1] "city_jjk"    "countryname" "longitude"    "latitude"
```

```
## [5] "geometry"
```

```
cities_sf %>% st_drop_geometry() %>% count(countryname, sort = TRUE) %>%  
  head(3)
```

```
## # A tibble: 3 x 2
```

```
##   countryname      n
```

```
##   <chr>          <int>
```

```
## 1 Italy          408
```

```
## 2 France         351
```

```
## 3 Spain          270
```

The geometry column is sticky. `select()` keeps it whether you ask for it or not. Use `st_drop_geometry()` on the rare occasion you do not want it.

Plotting what we loaded

```
par(mar = c(0, 0, 0, 0)) # no margins
plot(st_geometry(europe_sf), col = "grey93", border = "grey65",
      xlim = c(-11, 31.5), ylim = c(35, 71.5)) # drop the islands
plot(st_geometry(cities_sf), add = TRUE,
      pch = 20, cex = 0.35, col = "#005293")
```



run it

Plot early and plot often. Your eye catches a bad projection faster than any summary statistic will.

Reading points from CSV files

This is how `cities_sf` was built. Point data arrives as a CSV and must be made spatially aware.

```
cities_csv <- read_csv("../data/europe_cities_1801.csv",  
                      show_col_types = FALSE)  
cities_csv <- cities_csv %>%  
  filter(!(is.na(longitude) | is.na(latitude)))  
  
cities_sf <- st_as_sf(cities_csv, remove = FALSE,  
                    coords = c("longitude", "latitude"), crs = 4326)  
nrow(cities_sf)
```

```
## [1] 1801
```

- Longitude first, then latitude. Reversing them is a classic error, and it drops your European cities into the Indian Ocean.
- `remove = FALSE` keeps the two source columns instead of consuming them into the geometry. Default behavior is to consume them, and we want them back in an hour.

Sources for vector data

R packages

- `rnaturalearth`, countries, rivers, roads
- `tidycensus` and `tigris`, US census
- `osmdata`, OpenStreetMap
- `eurostat`, Eurostat data and maps

A few links

- freegisdata.rtwilson.com
- data.humdata.org
- Google dataset search

The data you can download is the data everyone has

Everything on this slide is data somebody else already made. Your paper wants the thing nobody has made yet. Keep that in view from now until Week 5.

Break

Ten minutes.

Projections, distance, and area

Watch it happen

```
nairobi <- st_as_sf(tibble(lon = 36.82241, lat = -1.287822),  
                    coords = c("lon", "lat"), crs = 4326)  
  
a <- nairobi %>% st_set_crs(3857)  
b <- nairobi %>% st_transform(3857)  
  
st_coordinates(a)  
st_coordinates(b)
```

st_set_crs() and **st_transform()** are not the same thing

st_set_crs() changes the label and leaves the coordinates alone, so it is only correct when the CRS was missing or wrong. **st_transform()** changes the coordinates. Use the first where you needed the second and your data ends up in the ocean.

So a claims to be in meters while still holding degrees. Neither R nor a language model will tell you which of the two you wanted.

A tale of two cities

```
cty1 <- tibble(name = "Nairobi", lon = 36.82241, lat = -1.287822) %>%  
  st_as_sf(coords = c("lon", "lat"), crs = 4326)  
  
cty2 <- tibble(name = "Kinshasa", x = 1531775, y = -531896.9) %>%  
  st_as_sf(coords = c("x", "y"), crs = "+proj=moll")  
  
st_crs(cty1)$epsg
```

```
## [1] 4326
```

```
st_is_longlat(cty2)
```

```
## [1] FALSE
```

Try to `rbind()` these two and R refuses, because the CRSs differ. It will not always be so helpful.

Always transform data in different CRSs

`st_transform()` interfaces with GDAL for all geographic transformations.

```
cty2 <- cty2 %>% st_transform(4326)
ctys <- rbind(cty1, cty2)
st_coordinates(ctys)
```

```
##           X           Y
## [1,] 36.82241 -1.287822
## [2,] 15.31035 -4.303362
```

Name projections by EPSG code, not by nickname

“Albers” is ambiguous. EPSG:5070 is not. A reader can look it up and so can you in six months.

Distances redux

- **Geographic coordinates.** `st_distance()` returns great circle distances.
- **Projected coordinates.** It returns Euclidean distances in native map units.

```
st_distance(cty1, cty2)
```

```
## Units: [m]  
##      [,1]  
## [1,] 2412295
```

- “Native map units” is the phrase that should make you check what those units are. That is the smaller of the two questions. The larger one is whether the projection suits the measurement at all, because a conformal projection preserves angles by distorting distance, and it does so most at high latitude.

Areas and length

- **Geographic coordinates.** `st_area()` gives geodetic areas.
- **Projected coordinates.** Areas in native map units.

```
europa_sf %>%  
  mutate(area_km2 = as.numeric(st_area(.)) / 1e6) %>%  
  st_drop_geometry() %>%  
  arrange(desc(area_km2)) %>%  
  select(CntryName, area_km2) %>%  
  head(4)
```

```
##   CntryName area_km2  
## 1    France 546758.2  
## 2    Spain 505638.6  
## 3    Sweden 443719.1  
## 4   Germany 356281.9
```

Use `st_length()` for line features. It behaves like `st_distance()`. And note which column we asked for. This shapefile also carries one called `name`, which holds the string “Cntry Name:” in every row, a header that was read in as data. It would have printed four times without complaint.

The area that is two and a half times too big

Germany, measured three ways. All three run without an error or a warning.

```
de <- europe_sf %>% filter(CntryName == "Germany")

as.numeric(st_area(de)) / 1e6           # 4326, geodetic
as.numeric(st_area(st_transform(de, 3035))) / 1e6   # equal-area
as.numeric(st_area(st_transform(de, 3857))) / 1e6   # Web Mercator
```

Conformal, not equal-area

Germany is about 357,600 square kilometers. The first two agree, at 356,282 and 357,576. Web Mercator returns **908,353**, two and a half times too big, in a number correctly labeled as square meters. Mercator preserves angles by inflating area, by roughly $\sec^2(\varphi)$. At Germany's latitude that factor is 2.53, and the observed error is 2.55.

Units would not have caught this. The number was labeled correctly and it was still wrong. Choose equal-area before `st_area()`, EPSG:3035 in Europe and EPSG:5070 in the United States. One projection used for everything is the anti-pattern.

An aside on units

Simple features report distances, lengths, and areas with their actual units.

```
library(units)
a_to_b <- st_distance(cty1, cty2)
set_units(a_to_b, km)
```

```
## Units: [km]
##      [,1]
## [1,] 2412.295
```

Units are a free verification layer. A number carrying [m] cannot be silently mistaken for degrees. Strip them with `as.numeric()` only when you must, and know what you are giving up when you do.

Distance as an instrument

From `st_distance()` to a research design

Everything so far has been mechanics. Here is why the mechanics matter.

Dittmar (2011), *Information Technology and Economic Change*

- Printing arrived in Mainz around 1450 and spread outward across Europe.
- Cities that adopted the press grew faster over the following century.
- The problem. Adoption was not random. Prosperous, connected cities plausibly got the press first and would have grown anyway.

The instrument is **distance from Mainz**, which is one `st_distance()` call away once the city points are loaded.

You already have everything you need

Mainz is one row of `cities_sf`, sitting in our data all evening.

```
mainz <- cities_sf %>% filter(city_jjk == "MAINZ")  
st_coordinates(mainz)
```

```
##           X           Y  
## [1,] 8.247253 49.99286
```

The instrument is one call to a function we met twenty minutes ago.

```
cities_sf <- cities_sf %>%  
  mutate(km_mainz = as.numeric(st_distance(., mainz)) / 1000)
```


Nearest neighbors of the press

```
cities_sf %>% st_drop_geometry() %>%  
  arrange(km_mainz) %>%  
  transmute(city_jjk, countryname, km = round(km_mainz)) %>%  
  slice(2:6)
```

```
## # A tibble: 5 x 3  
##   city_jjk      countryname      km  
##   <chr>        <chr>          <dbl>  
## 1 WIESBADEN    Germany          8  
## 2 OPPENHEIM    Germany         18  
## 3 BINGEN       Germany         25  
## 4 BADKREUZNACH Germany         31  
## 5 DARMSTADT    Germany         32
```

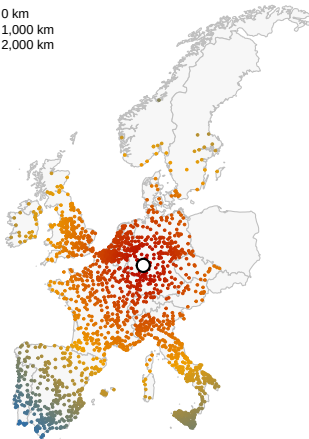
Mainz itself is row one at zero kilometers, so we start at two.

The instrument, drawn

Distance from Mainz, 1,801 cities

from Mainz

- 0 km
- 1,000 km
- 2,000 km



Red is near, pale is far. Printing spread outward from that white dot, and so, Dittmar argues, did a shock to city growth.

run it

What it cost you to build

Two lines

`filter()` to find Mainz, `st_distance()` to measure. Both of them functions you did not know at 7:20 this evening and now do.

- That is the whole mechanical content of a famous instrument in a top-five paper.
- A language model would have written those two lines in about four seconds, and correctly.

Which raises the obvious question, and it is the question the rest of the course answers. If building the instrument is nearly free, where did the paper go?

The design, written down

For city i , let Y_i be log population growth from 1500 to 1600, let D_i be one if the city had a press by 1500, and let Z_i be distance from Mainz. X_i holds log population in 1500.

$$\text{second stage} \quad Y_i = \beta D_i + X_i' \gamma + \varepsilon_i$$

$$\text{first stage} \quad D_i = \pi Z_i + X_i' \delta + \nu_i$$

$$\text{reduced form} \quad Y_i = \rho Z_i + X_i' \lambda + u_i$$

With one instrument for one endogenous regressor the estimate is a ratio, $\hat{\beta}_{IV} = \hat{\rho} / \hat{\pi}$. Nothing in that expression is hard. Every term is a coefficient from a regression you have run since your first year. **Relevance** is $\pi \neq 0$ and you can test it. **Exclusion** is $E[Z_i \varepsilon_i] = 0$ and you cannot.

The first stage

Press adoption dates and city populations, joined onto the points we already have.

```
press <- read_csv("../data/europe_press_1500.csv",
                  show_col_types = FALSE)

est <- cities_sf %>% st_drop_geometry() %>%
  filter(!city_jjk %in% c("ALBA", "HALLE")) %>% # one name, two cities
  inner_join(press, by = "city_jjk") %>%
  filter(pop1500 > 0, pop1600 > 0) %>% # log(0) is -Inf, not NA
  mutate(press1500 = as.integer(!is.na(print_date) & print_date <= 1500),
         growth     = log(pop1600) - log(pop1500))

fs <- lm(press1500 ~ km_mainz + log(pop1500), data = est)
round(summary(fs)$coefficients, 6)
```

```
##              Estimate Std. Error  t value Pr(>|t|)
## (Intercept) -0.057804   0.034626 -1.669390 0.095462
## km_mainz    -0.000138  0.000027 -5.203517 0.000000
## log(pop1500) 0.224095   0.014518 15.435231 0.000000
```

A hundred kilometers closer to Mainz raises the probability of a press by about 1.4 points, with $t = -5.2$. **Relevance holds.** Dittmar's instrument is doing exactly what he says.

Now estimate the thing you care about

```
ols <- lm(growth ~ press1500 + log(pop1500), data = est)
rf <- lm(growth ~ km_mainz + log(pop1500), data = est)

c(OLS = coef(ols)["press1500"],          # both estimates
  IV = coef(rf)["km_mainz"] / coef(fs)["km_mainz"])
```

```
## OLS.press1500    IV.km_mainz
##      0.1417409      -0.4542405
```

```
library(AER)                                     # IV again, with its SE
iv <- ivreg(growth ~ press1500 + log(pop1500) | km_mainz + log(pop1500),
            data = est)
round(summary(iv)$coefficients["press1500", ], 3)
```

```
##      Estimate Std. Error      t value      Pr(>|t|)
##      -0.454      0.252      -1.799      0.072
```

One of these is an answer and the other is a shrug

Read the standard error, not just the estimate. It puts the instrument's interval at -0.95 to $+0.04$, which excludes neither zero nor a large negative effect. The instrument did not overturn OLS. It declined to confirm it, at a heavy price in precision.

Where is the variation actually coming from?

Distance from Mainz is a smooth function of where a city sits. So ask what is left of it once you let the regression know where that is.

```
f1 <- lm(press1500 ~ km_mainz + log(pop1500), data = est)
f2 <- update(f1, . ~ . + longitude + latitude)
f3 <- update(f1, . ~ . + poly(longitude, 2) + poly(latitude, 2))

round(sapply(list(f1, f2, f3),
                function(m) summary(m)$coefficients["km_mainz", "t value"]^2), 1)
## [1] 27.1 15.2 0.0
```

The instrument is strong, then weak, then gone. Dittmar (2011) conditions on a polynomial in latitude and longitude, so whatever identifies his estimate is what survives the third column. Ours rides the raw continental gradient, which is the first. **We are not computing his estimate badly. We are using different variation.**

Which of those two numbers do you believe?

The exclusion restriction

$E[Z_i \varepsilon_i] = 0$. Distance from Mainz must move city growth only through the press, and through nothing else whatsoever.

- Relevance held decisively, and then dissolved the moment we controlled for position. It was never the hard part, and it was never the whole story.
- But distance from Mainz is also distance from the Rhine, from the center of the Holy Roman Empire, from the trade routes of the Low Countries, and from the densest urban network in fifteenth century Europe. Each has its own claim on growth between 1500 and 1600, and in our two-line specification every one of them sits inside ε_i .
- **So the sign flip is not a finding.** It is what an instrument does when the exclusion restriction is carrying weight it has not earned.

Dittmar (2011) does not run what we just ran. He instruments with log distance, restricts the sample, and conditions on a control set including a polynomial in latitude and longitude, which absorbs much of what we left in ε_i . Ours is deliberately the naive version, and the gap between the two is the forty pages.

The biggest move came from a table note

We went back to [Dittmar \(2011\)](#) and rebuilt his specification on our data, one choice at a time.

##	IV	n
## ours, linear distance	-0.454	744
## + his sample restriction	-0.235	430
## + his controls	-0.724	430
## Dittmar (2011) Table VII	0.580	410

The largest single move is the middle row, and it is not a modelling choice. It is one sentence in his table note restricting the sample to cities observed 1500 to 1800. Adding his controls then triples the standard error, so the specification closest to his is also the one that identifies least. **We never reach his number, and we can say exactly where we lose it.** The print cities in our data grew nine percentage points faster than the rest. He reports twenty.

The lesson for your own work

Easy to compute, hard to defend

Geography makes instruments easy to compute and hard to defend. The distance was two lines and you watched them run. The exclusion restriction is the other forty pages.

- This asymmetry will get worse, not better. A model will happily generate any geographic instrument you describe, in seconds.
- What it cannot do is know whether the exclusion restriction holds in your setting. That judgment is yours, and it is what you are actually being trained in.

Tonight in one slide

1. R as a GIS, because a script is reproducible and auditable and a click is neither.
2. The tidyverse gives us verbs. `sf` makes spatial data a tibble with a geometry column.
3. Check the CRS before you measure. `st_set_crs()` relabels, `st_transform()` moves.
4. Check `nrow()` around every join. Units are a free verification layer.
5. Distance is easy to compute and hard to defend as an instrument.

Before next week

For Week 3, Buffers and Regression Discontinuity

1. Read [Dell \(2010\)](#) on the Peruvian mita.
2. Work through tonight's script yourself, by hand. Nobody is stopping you from prompting it. The Week 6 lab exam is.
3. Bring one question about your own paper's data.

Reminder. The Week 6 lab exam is closed book and closed model, and covers exactly this material. Tonight is the kind of thing it asks about.

References I

Dell, M. (2010). The persistent effects of peru's mining mita. *Econometrica*, 78(6):1863–1903.

Dittmar, J. E. (2011). Information technology and economic change: the impact of the printing press. *The Quarterly Journal of Economics*, 126(3):1133–1172.